School of Computer Science Engineering & Information Systems

Department of Software & Systems Engineering

ISWE406P – Agile Development Process and Devops Lab

ASSESSMENT -4

NAME : SARAVANAN.R

REGISTER NO : 23MIS0460

FACULTY NAME : DR . RAMALINGAM M

EXPERIMENT – 1 :

Spring Boot CI/CD Pipeline using GitHub, Jenkins, Docker, and Kubernetes

Objective:

To implement a complete CI/CD pipeline using Jenkins that builds a Spring Boot application, containerizes it using Docker, and deploys it on Kubernetes automatically using GitHub integration.

Tools Used:

- Jenkins Automation Server
- Git & GitHub
- Java 17
- Apache Maven
- Docker Desktop
- Kubernetes (enabled in Docker Desktop)
- Web Browser
- Apache Maven
- Java JDK

- Web Browser

<u>Step 1:</u> Install Required Tools

- ❖ Install **Java 17** and verify:

java -version

- ❖ Install **Apache Maven** and verify:

mvn -version

- ❖ Install **Docker Desktop** and enable Kubernetes.
- ❖ Install **Jenkins**.
- ❖ Install **Git**.

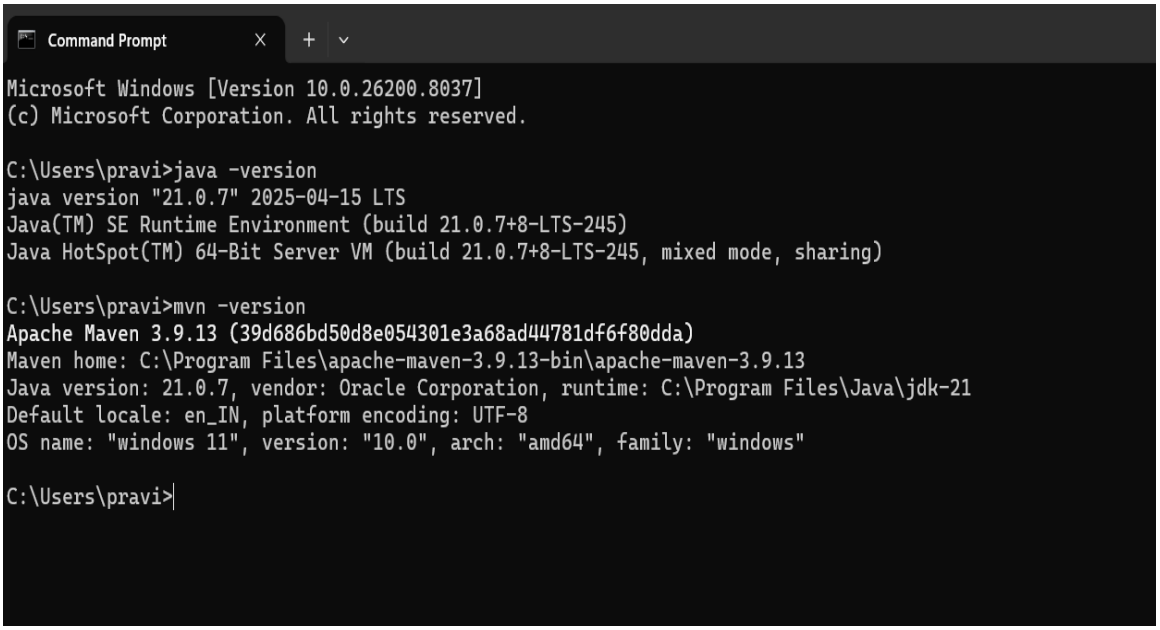

Figure 1: Installed Tools Verification (Java, Maven, Docker)

<u>Step 2:</u> Create Spring Boot Project

<u>pom.xml (Important Dependencies):</u>

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
</parent>

<dependencies>
```

```
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

Main Application (App.java) :

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Controller (HelloController.java) :

```
@RestController
public class HelloController {
    @GetMapping("/")
    public String home() {
        return "App Running";
    }
}
```

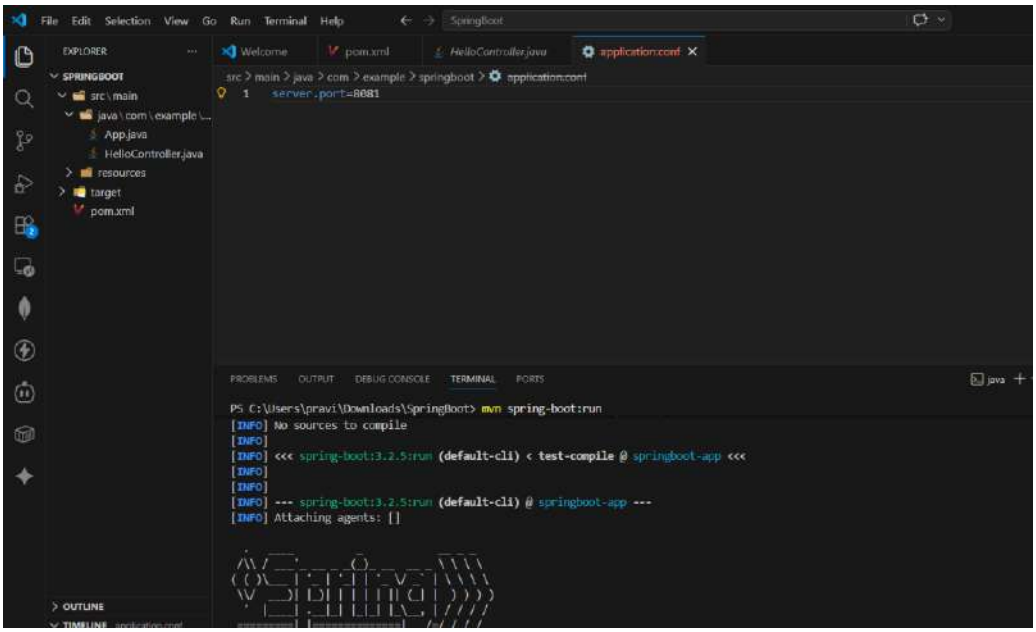

Figure 2: Spring Boot Project Structure and Code

<u>Step 3:</u> Run Application Locally

- **Execute the command:**

mvn spring-boot:run

- **Open browser:**

  http://localhost:8080

- **Expected Output:**

  App Running

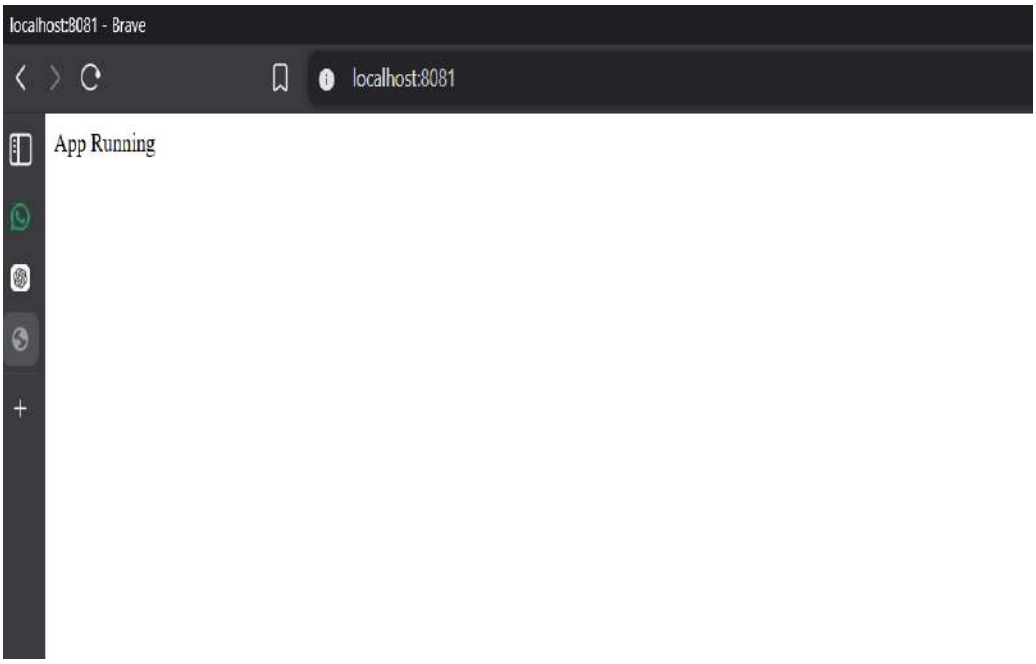

Figure 3: Spring Boot Application Running in Browser

Step 4: Docker Configuration

Create Dockerfile

```
FROM eclipse-temurin:17-jdk

WORKDIR /app

COPY target/app.jar app.jar

ENTRYPOINT ["java","-jar","app.jar"]
```

Build Docker Image

```
docker build -t springboot-app .
```

Run Container

```
docker run -p 8080:8080 springboot-app
```

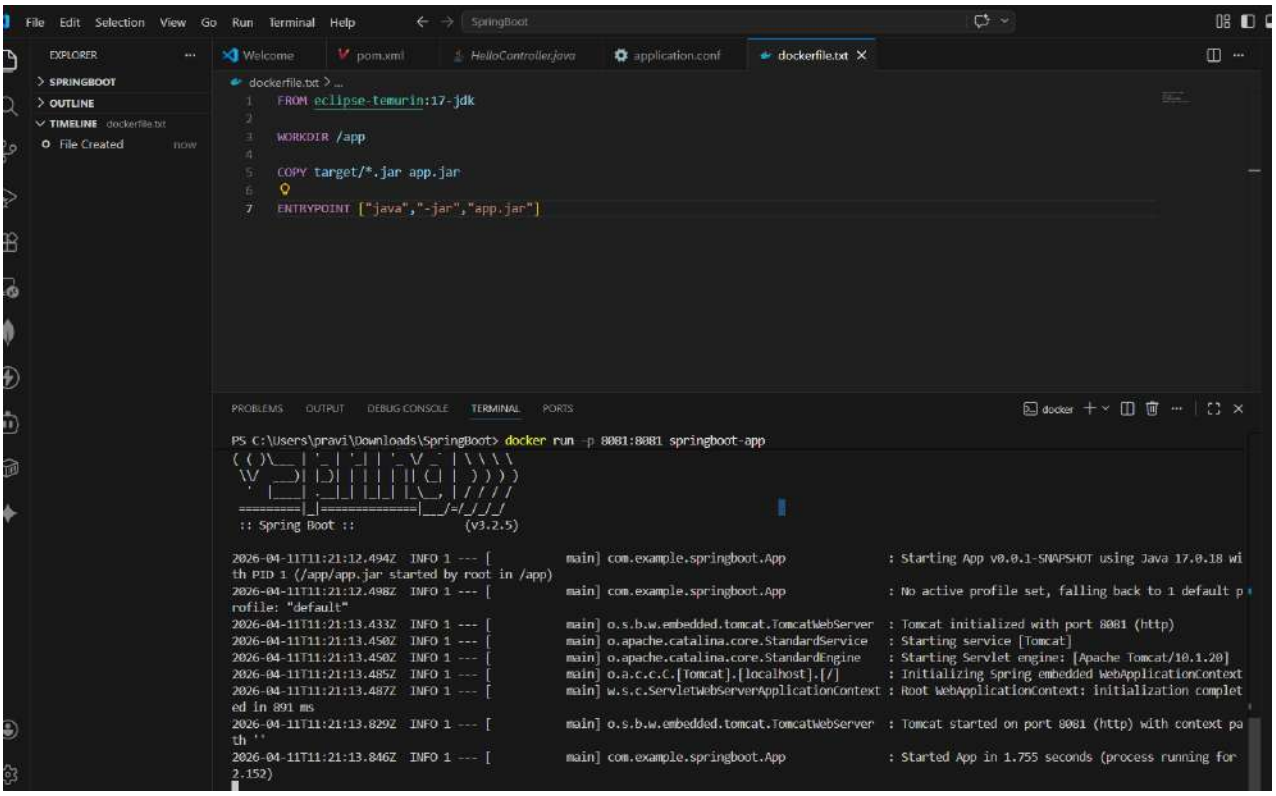

Figure 4: Docker Image Build and Container Execution

Step 5.1 Enable Kubernetes

- **Open Docker Desktop**

  1. Go to **Settings** ⚙
  2. Click **Kubernetes**
  3. Enable:
     ✓ Enable Kubernetes
  4. Click **Apply & Restart**

⏳ Wait until:

Kubernetes is running ☑

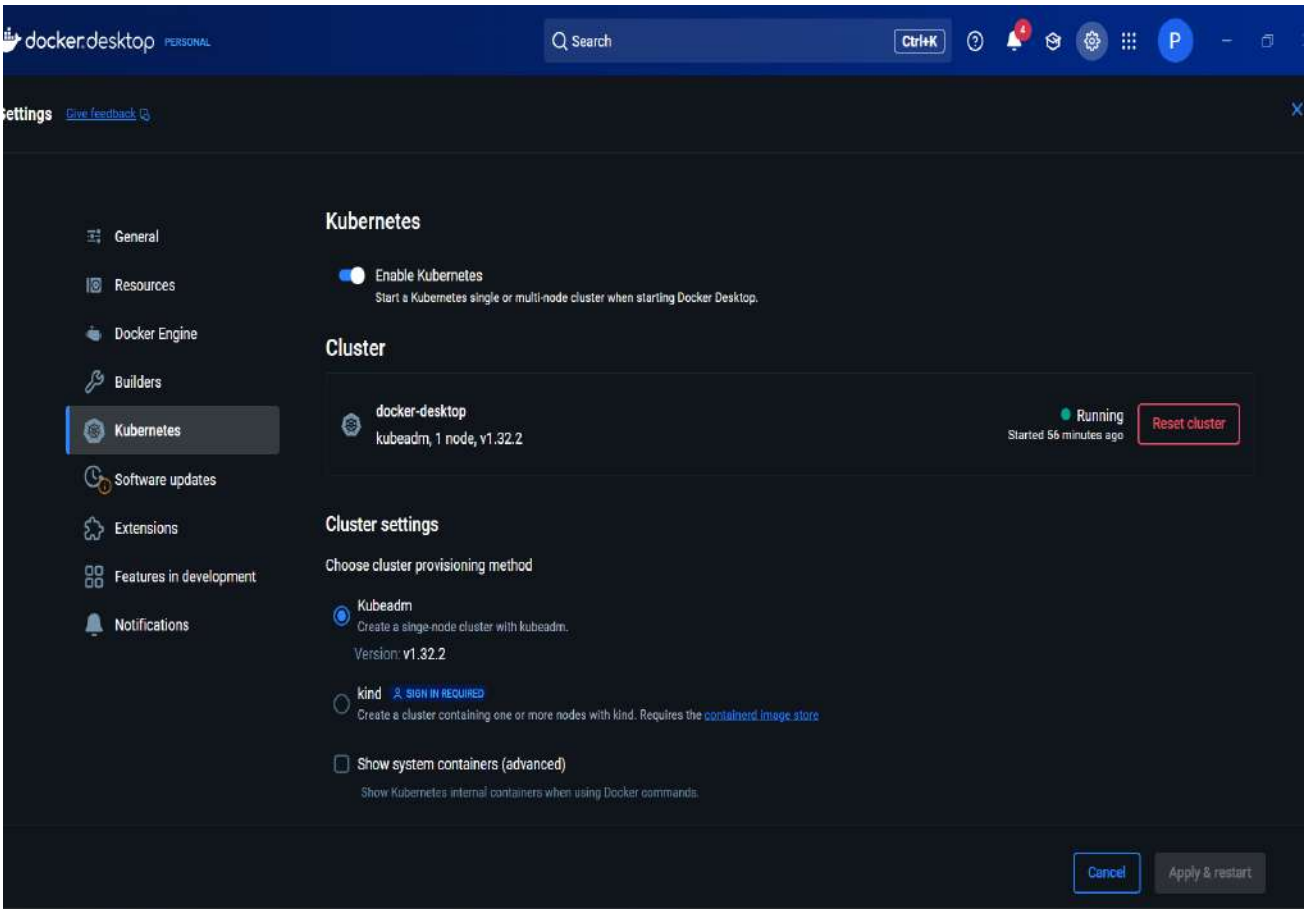

Figure 5: Kubernetes Enabled in Docker Desktop

📄 Step 5.2: Create deployment.yaml

👉 In VS Code:

1. Right-click project → **New File**
2. Name:

deployment.yaml

CODE :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: springboot
  template:
    metadata:
      labels:
        app: springboot
    spec:
```

```
containers:
- name: app
  image: springboot-app
  imagePullPolicy: Never
  ports:
  - containerPort: 8081
```

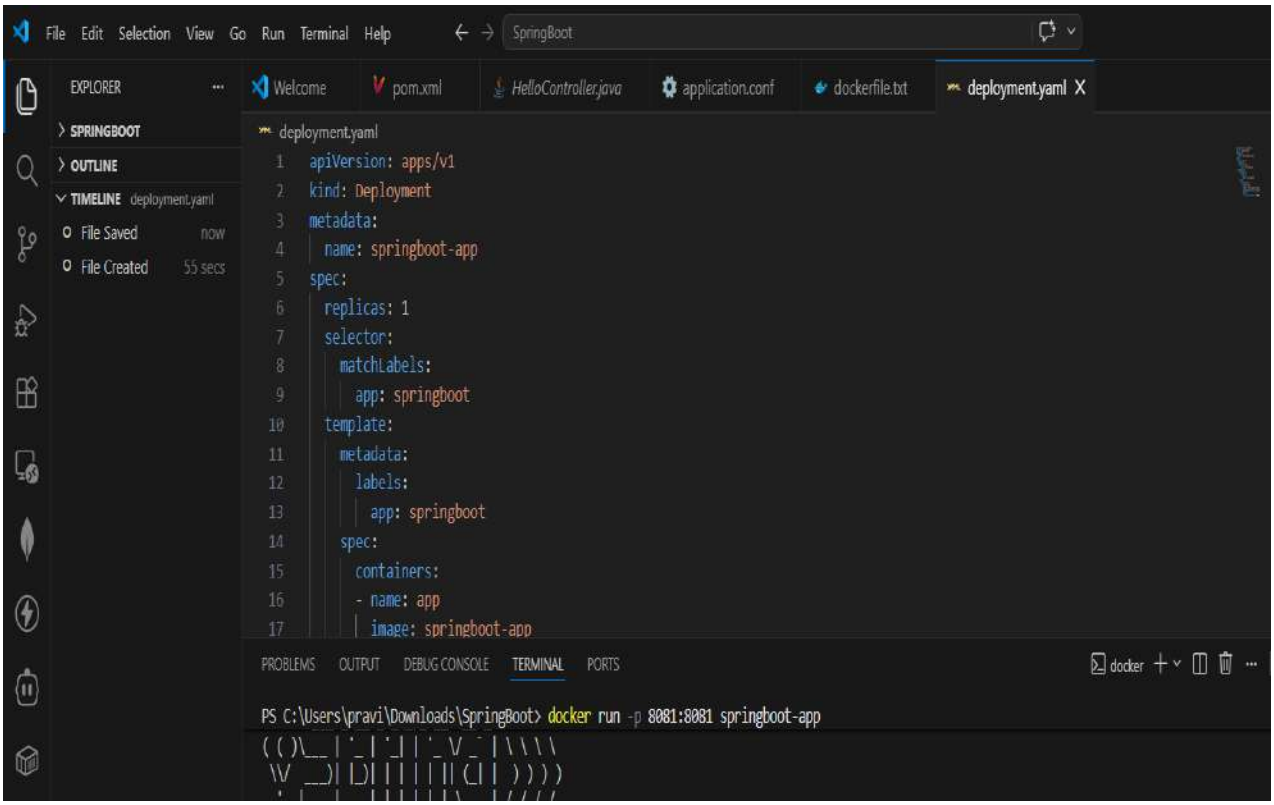

Figure 6: deployment.yaml File Created

IMPORTANT

👉 Your Docker image must already exist:

Run to check:

docker images

👉 You should see:

springboot-app

Step 5.3: Deploy to Kubernetes

👉 Open terminal in project folder:

cd C:\Users\sarva\Downloads\SpringBoot

👉 Run:

kubectl apply -f deployment.yaml

☑ **Expected output:**

deployment.apps/springboot-app created

```
15      containers:
16      - name: app
17        image: springboot-app
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\pravi\Downloads\SpringBoot> ^C
PS C:\Users\pravi\Downloads\SpringBoot> kubectl apply -f deployment.yaml
deployment.apps/springboot-app created
```

Figure 7: Kubernetes Deployment Created

Step 5.4: Check Pods

Run:

kubectl get pods

Expected output:

springboot-app-xxxxx Running

```
PS C:\Users\pravi\Downloads\SpringBoot> kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
springboot-app-5bbc76c4fc-s9k24   1/1     Running   0          11s
```

Figure 8: Kubernetes Pod Running

Step 5.5: Access Application

Kubernetes needs port forwarding:

Run:

kubectl port-forward deployment/springboot-app 8082:8081

Open browser:

http://localhost:8082

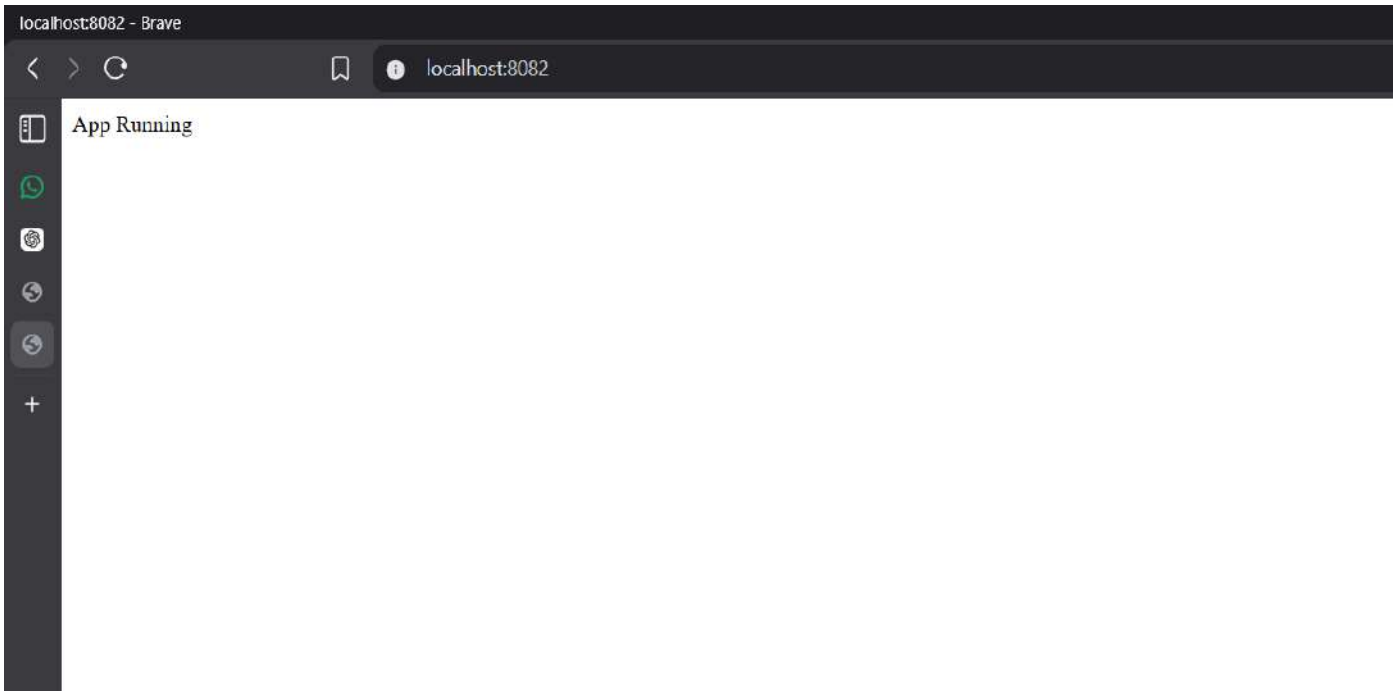

Figure 9: Application Running via Kubernetes

☑ **Output:**

App Running

<u>Step 6:</u> Push Project to GitHub

Procedure:

- Open terminal in project folder:

```
git init
git add .
git commit -m "Initial commit"
```

- Create repository in GitHub and connect:

```
git branch -M main
git remote add origin https://github.com/<your-username>/springboot-app.git
git push -u origin main
```

Expected Outcome:

- Project files are uploaded to GitHub repository
- Repository contains:

- src/
- pom.xml
- Dockerfile
- deployment.yaml

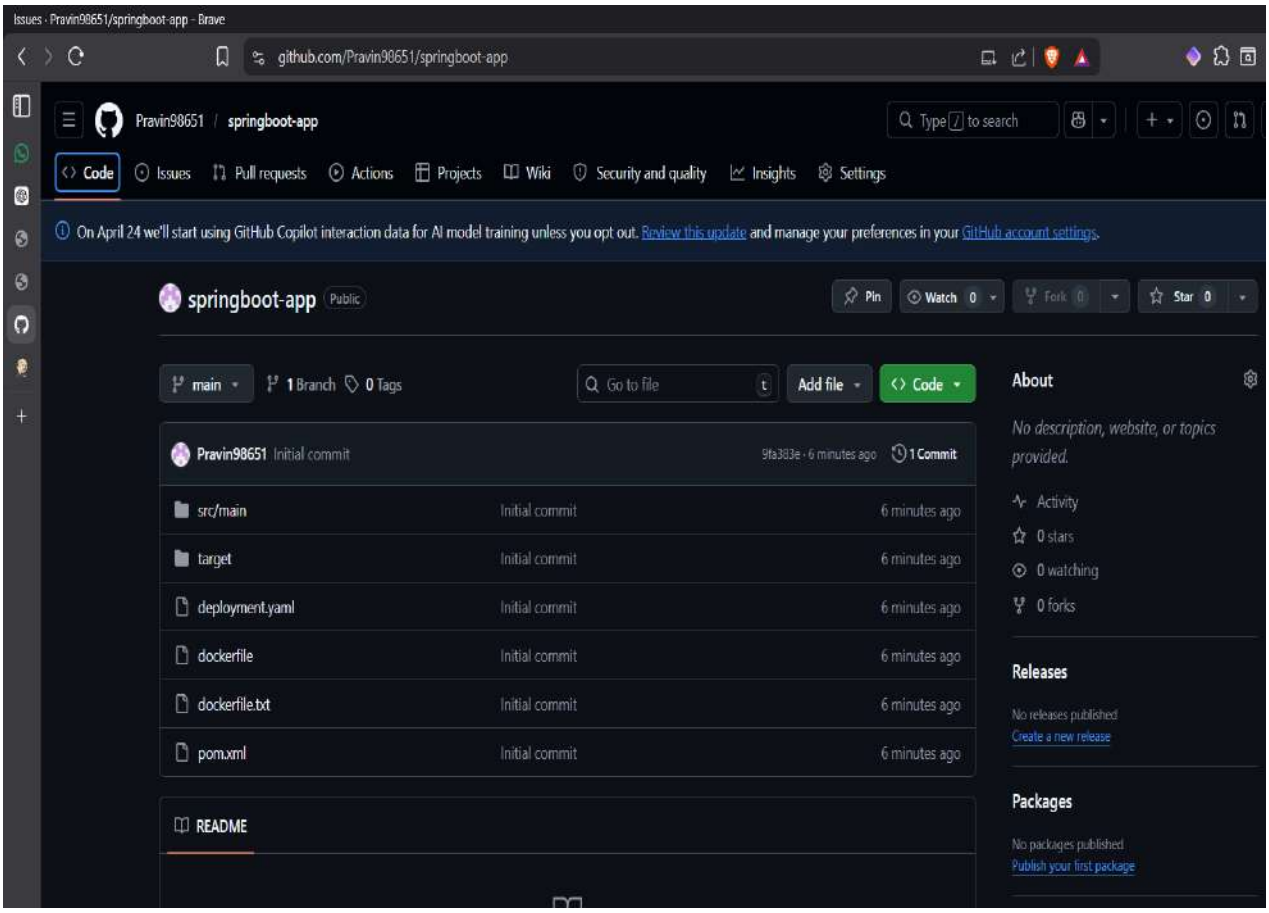

Figure 10: GitHub Repository with Project Files

<u>Step 7:</u> Create Jenkins Pipeline

Step 7.1: Create Jenkinsfile

- In project root, create file:

Jenkinsfile

Jenkinsfile Code:

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "springboot-app"
    }

    stages {
```

```
        stage('Clone') {
            steps {
                git 'https://github.com/<your-username>/springboot-app.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Docker Build') {
            steps {
                sh 'docker build -t $IMAGE_NAME .'
            }
        }

        stage('Deploy to Kubernetes') {
            steps {
                sh 'kubectl apply -f deployment.yaml'
            }
        }
    }
}
```

Explanation:

- Clone → Fetch code from GitHub
- Build → Compile using Maven
- Docker → Create container image
- Deploy → Deploy to Kubernetes

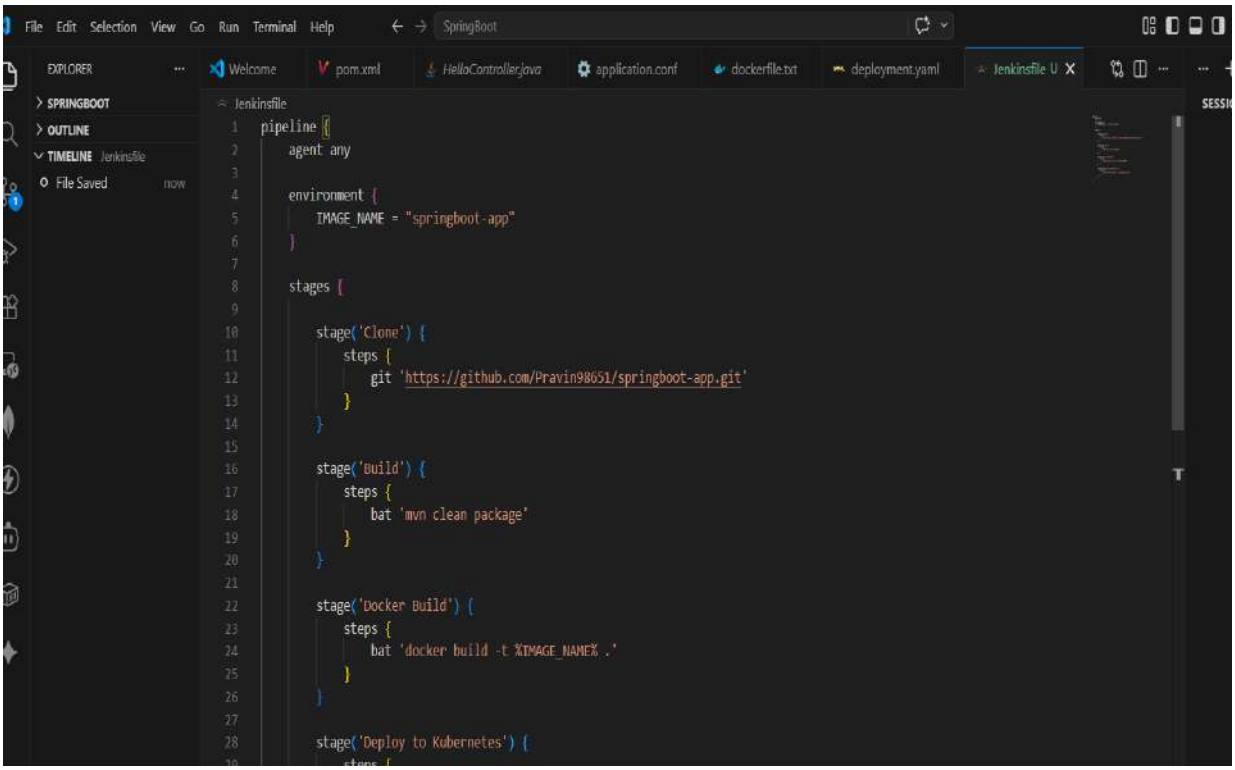

Figure 11: Jenkinsfile Pipeline Script

<u>Step 8:</u> Configure Jenkins Job

Procedure:

1. Open Jenkins Dashboard
2. Click **New Item**
3. Enter name → Select **Pipeline**
4. Click **OK**

Configure:

- Under Pipeline:
  - Select **Pipeline from SCM**
  - SCM: Git
  - Repository URL:

https://github.com/<your-username>/springboot-app.git

  - Script Path:

Jenkinsfile

Save and Build

Click:

Build Now

Expected Outcome:

- Jenkins executes pipeline stages:
    - Clone
    - Build
    - Docker
    - Deploy

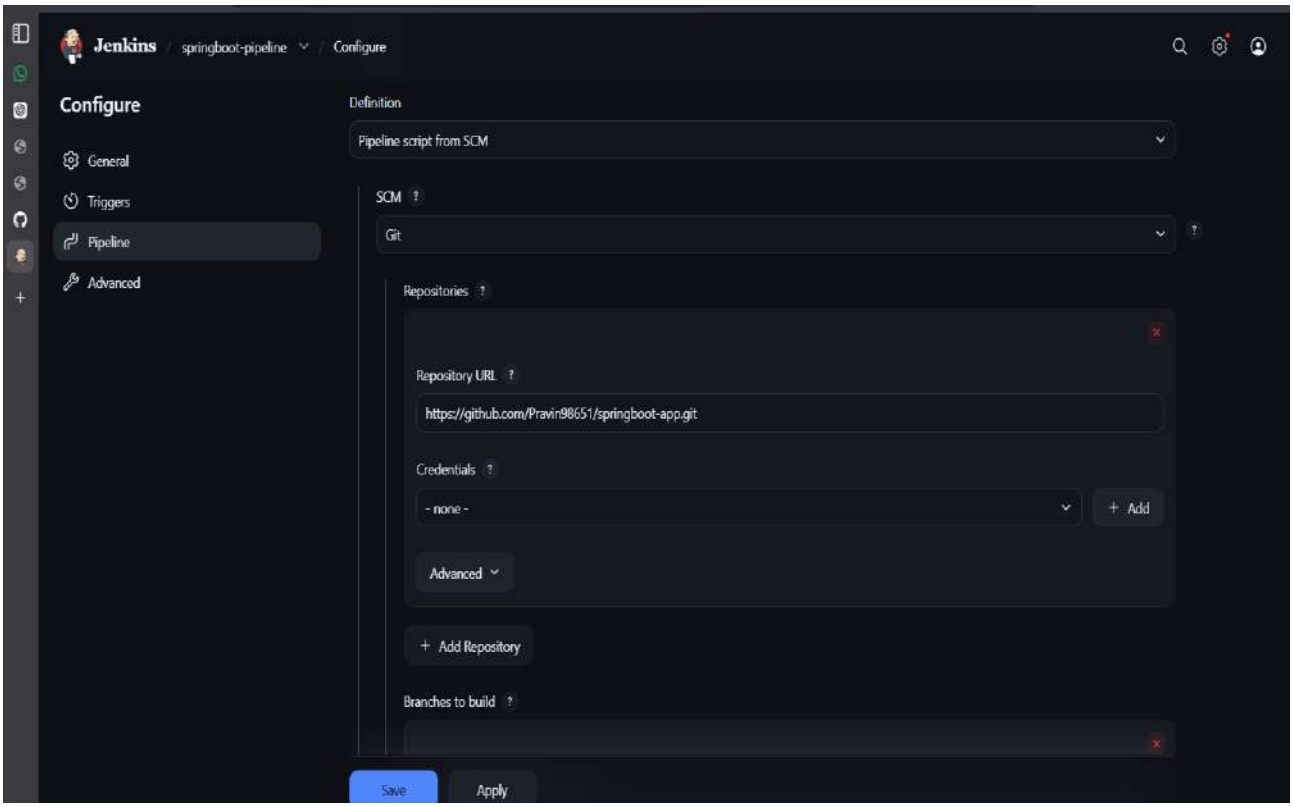

Figure 12: Jenkins Pipeline Configuration

<u>Step 9:</u> GitHub Webhook Integration

Objective:

To trigger Jenkins automatically on every code push.

Procedure:

1. Go to GitHub repository
2. Click **Settings** → **Webhooks**
3. Click **Add Webhook**

Configure:

- Payload URL:

http://<your-system-ip>:8080/github-webhook/

- Content type:

application/json

- Events:

Just the push event

Save Webhook

Expected Outcome:

- Every push triggers Jenkins automatically

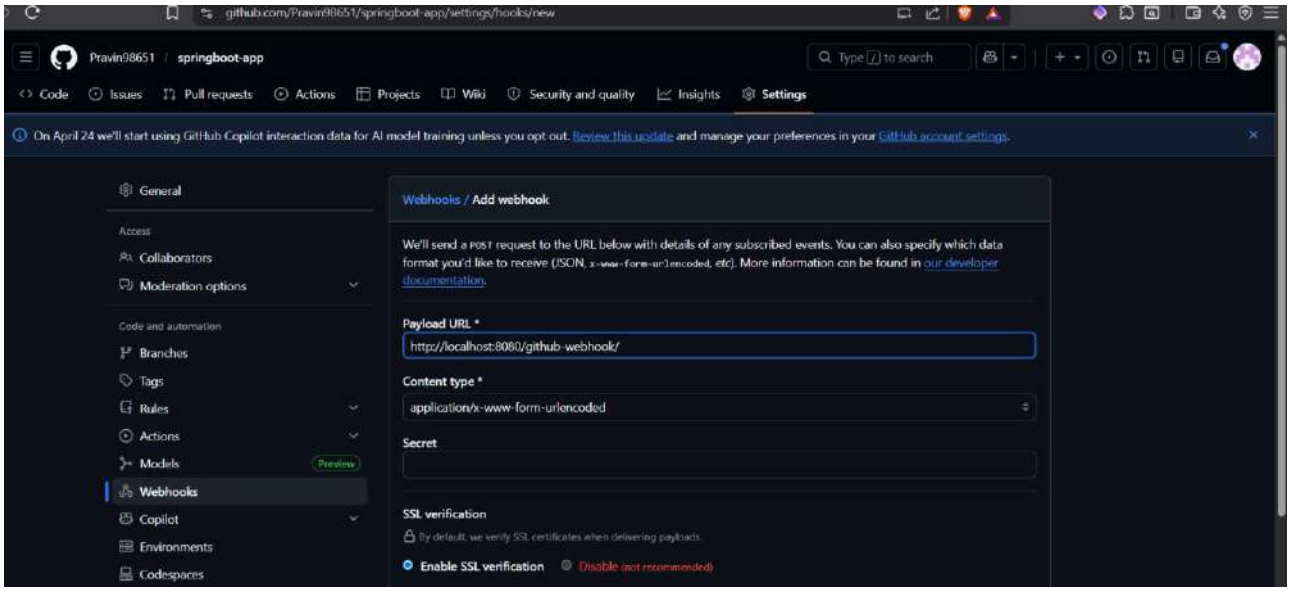

Figure 13: GitHub Webhook Configuration

<u>Step 10:</u> Automatic CI/CD Execution

Procedure:

Make a small code change:

```
git add .
git commit -m "Test CI/CD pipeline"
git push
```

What Happens Automatically:

1. Code pushed to GitHub
2. Webhook triggers Jenkins
3. Jenkins pipeline starts
4. Application is rebuilt

5. Docker image updated
6. Kubernetes deployment updated

Verification:

kubectl get pods

Application will restart with new changes.

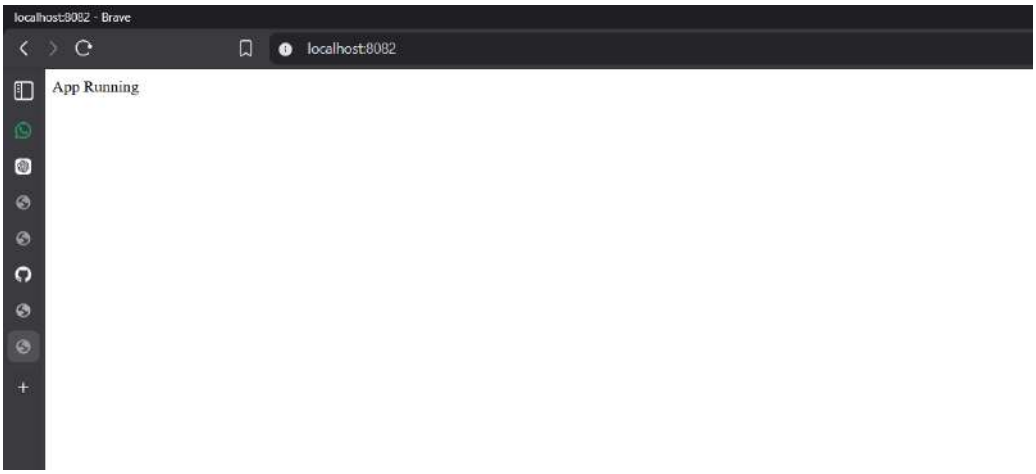

Figure 14: Updated Application Running

Final Result

The CI/CD pipeline is successfully implemented with:

- Continuous Integration using Jenkins
- Continuous Deployment using Kubernetes
- Version control using GitHub
- Containerization using Docker

Conclusion

A complete automated CI/CD pipeline has been developed where any code change pushed to GitHub automatically triggers Jenkins to build, containerize, and deploy the application to Kubernetes, ensuring faster and reliable software delivery.